

# Multivariate Normal Distribution

---

## Table of Contents

---

1. [Introduction](#)
  2. [From 1D to Multivariate](#)
  3. [Mean Vector and Covariance Matrix](#)
  4. [Probability Density Function](#)
  5. [Properties](#)
  6. [Parameter Estimation](#)
  7. [Implementation in Python](#)
  8. [Practical Examples](#)
- 

## Introduction

---

The **multivariate normal distribution** (also called multivariate Gaussian) is the generalization of the 1D normal distribution to multiple dimensions.

### Why it matters:

- Models correlated random variables
  - Foundation of many machine learning algorithms
  - Appears naturally due to Central Limit Theorem
  - Analytically tractable (nice mathematical properties)
-

# From 1D to Multivariate

---

## 1D Normal Distribution

PDF:

$$f(x) = (1/\sqrt{2\pi\sigma^2}) \exp(-(x-\mu)^2/(2\sigma^2))$$

Parameters:

- $\mu$ : mean (location)
- $\sigma^2$ : variance (spread)

## 2D Normal Distribution

PDF:

$$f(x, y) = (1/(2\pi\sqrt{|\Sigma|})) \exp(-\frac{1}{2} [x-\mu]^T \Sigma^{-1} [x-\mu])$$

where:

- $\mathbf{x} = [x, y]^T$  (2D point)
- $\boldsymbol{\mu} = [\mu_x, \mu_y]^T$  (mean vector)
- $\boldsymbol{\Sigma} = 2 \times 2$  covariance matrix

## d-Dimensional Normal Distribution

PDF:

$$f(\mathbf{x}) = (1/\sqrt{((2\pi)^d |\Sigma|)}) \exp(-\frac{1}{2} (\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}-\boldsymbol{\mu}))$$

where:

- $\mathbf{x} \in \mathbb{R}^d$  (d-dimensional vector)

- $\boldsymbol{\mu} \in \mathbb{R}^d$  (mean vector)
- $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$  (covariance matrix)

## Mean Vector and Covariance Matrix

### Mean Vector $\boldsymbol{\mu}$

The **mean vector** specifies the center of the distribution:

$$\boldsymbol{\mu} = [\mu_1, \mu_2, \dots, \mu^d]^T = E[X]$$

Each component is the expected value of that dimension:

- $\mu_1 = E[X_1]$
- $\mu_2 = E[X_2]$
- etc.

**Example (2D):**

```
mu = np.array([6, 6]) # Center at (6, 6)
```

### Covariance Matrix $\boldsymbol{\Sigma}$

The **covariance matrix** describes spread and correlation:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \dots & \sigma_{1^d} \\ \sigma_{21} & \sigma_{22} & \dots & \sigma_{2^d} \\ \dots & \dots & \dots & \dots \\ \sigma^{d1} & \sigma^{d2} & \dots & \sigma^{dd} \end{bmatrix}$$

where:

- **Diagonal elements  $\sigma_{ii}$ :** Variance of  $X_i$
- **Off-diagonal  $\sigma_{ij}$ :** Covariance between  $X_i$  and  $X_j$

#### Properties:

1. **Symmetric:**  $\Sigma = \Sigma^T$  (since  $\text{Cov}(X,Y) = \text{Cov}(Y,X)$ )
2. **Positive semi-definite:**  $x^T \Sigma x \geq 0$  for all  $x$

## Understanding Covariance

#### Variance (diagonal):

$$\text{Var}(X) = E[(X - E[X])^2]$$

#### Covariance (off-diagonal):

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

#### Interpretation:

- $\text{Cov}(X,Y) > 0$ :  $X$  and  $Y$  tend to increase together (positive correlation)
- $\text{Cov}(X,Y) < 0$ : When  $X$  increases,  $Y$  tends to decrease (negative correlation)
- $\text{Cov}(X,Y) = 0$ : No linear relationship (uncorrelated)

## 2D Example

```
# Mean at (6, 6)
mu = np.array([6, 6])

# Covariance matrix
Sigma = np.array([[1.0, 0.6],    # Var(X)=1.0, Cov(X,Y)=0.6
                  [0.6, 1.0]])  # Cov(Y,X)=0.6, Var(Y)=1.0
```

#### What does this mean?

- Both  $X$  and  $Y$  have variance 1.0 (spread)

- X and Y are positively correlated (0.6)
- When X is above its mean, Y tends to be above its mean too

## Correlation vs Covariance

**Correlation coefficient:**

$$\rho(X, Y) = \text{Cov}(X, Y) / (\sigma_X \sigma_Y)$$

Range:  $-1 \leq \rho \leq 1$

**Building  $\Sigma$  from correlation:**

```
# Given: variances and correlation
var_x = 1.0
var_y = 1.0
rho = 0.6

# Covariance
cov_xy = rho * np.sqrt(var_x * var_y) # = 0.6

# Covariance matrix
Sigma = np.array([[var_x, cov_xy],
                  [cov_xy, var_y]])
```

## Probability Density Function

### Mathematical Form

For  $\mathbf{x} \in \mathbb{R}^d$ :

$$f(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (1/\sqrt{((2\pi)^d |\boldsymbol{\Sigma}|)}) \exp(-\frac{1}{2} (\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}-\boldsymbol{\mu}))$$

**Components:**

1. **Normalization:**  $1/\sqrt{(2\pi)^d |\Sigma|}$

- Ensures  $\int f(x) dx = 1$
- $|\Sigma|$  = determinant of covariance matrix

2. **Exponential term:**  $\exp(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu))$

- $(x-\mu)$ : deviation from mean
- $\Sigma^{-1}$ : inverse covariance (precision matrix)
- Quadratic form: measures "distance" accounting for correlation

**Log-PDF (More Numerically Stable)**

$$\begin{aligned} \log f(x) &= -\frac{1}{2} \log((2\pi)^d |\Sigma|) - \frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu) \\ &= \text{constant} - \frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu) \end{aligned}$$

**Why log?**

- Avoids numerical underflow (probabilities can be tiny)
- Converts products to sums
- Easier to work with in optimization

## Computing PDF in Python

```
from scipy.stats import multivariate_normal

# Parameters
mu = np.array([6, 6])
Sigma = np.array([[1.0, 0.6],
                  [0.6, 1.0]])

# Create distribution
mvn = multivariate_normal(mean=mu, cov=Sigma)

# Evaluate PDF at a point
x = np.array([6.5, 6.5])
pdf_value = mvn.pdf(x)
log_pdf_value = mvn.logpdf(x)

print(f"f({x}) = {pdf_value:.6f}")
print(f"log f({x}) = {log_pdf_value:.6f}")
```

## Visualizing 2D Normal PDF

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Create grid
x = np.linspace(0, 12, 100)
y = np.linspace(0, 12, 100)
X, Y = np.meshgrid(x, y)

# Evaluate PDF at each grid point
pos = np.dstack((X, Y))
Z = mvn.pdf(pos)

# 3D plot
fig = plt.figure(figsize=(12, 5))

ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X, Y, Z, cmap='viridis')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.set_zlabel('PDF')
ax1.set_title('3D PDF')

# Contour plot
ax2 = fig.add_subplot(122)
ax2.contourf(X, Y, Z, levels=20, cmap='viridis')
ax2.set_xlabel('X')
ax2.set_ylabel('Y')
ax2.set_title('Contour Plot')
ax2.axis('equal')

plt.tight_layout()
plt.show()
```

# Properties

---

## Property 1: Marginal Distributions are Normal

If  $\mathbf{X} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ , then each component  $X_i \sim N(\mu_i, \sigma_{ii})$

**Example:**

```
# 2D normal
mu = np.array([6, 6])
Sigma = np.array([[1.0, 0.6],
                  [0.6, 1.0]])

# Marginal distribution of X
mu_x = mu[0] # = 6
sigma_x = Sigma[0, 0] # = 1.0
# X ~ N(6, 1.0)

# Marginal distribution of Y
mu_y = mu[1] # = 6
sigma_y = Sigma[1, 1] # = 1.0
# Y ~ N(6, 1.0)
```

## Property 2: Linear Combinations are Normal

If  $\mathbf{X} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$  and  $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$ , then:

$$\mathbf{Y} \sim N(\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^T)$$

**Example:** Rotation

```
# Rotate 2D normal by 45 degrees
theta = np.pi / 4
A = np.array([[np.cos(theta), -np.sin(theta)],
               [np.sin(theta), np.cos(theta)]])

mu_rotated = A @ mu
Sigma_rotated = A @ Sigma @ A.T
```

### Property 3: Independence $\Leftrightarrow$ Zero Covariance

For normal distributions:

- $X_i$  and  $X_j$  are independent  $\Leftrightarrow \text{Cov}(X_i, X_j) = 0$

(This is special to normal distributions! Generally uncorrelated  $\neq$  independent)

**Example:** Diagonal covariance  $\rightarrow$  independent components

```
Sigma_diagonal = np.array([[1.0, 0.0],
                             [0.0, 1.0]])

# X and Y are independent!
```

### Property 4: Conditional Distributions are Normal

Given  $\mathbf{X} \sim N(\mu, \Sigma)$ , the conditional distribution is also normal.

For 2D case, if  $\mathbf{X} = [X_1, X_2]$ :

$$X_2 \mid X_1=x_1 \sim N(\mu_2 + (\sigma_{12}/\sigma_{11})(x_1 - \mu_1), \sigma_{22} - \sigma_{12}^2/\sigma_{11})$$

### Property 5: Sum of Independent Normals is Normal

If  $\mathbf{X} \sim N(\mu_x, \Sigma_x)$  and  $\mathbf{Y} \sim N(\mu_y, \Sigma_y)$  are independent:

$$\mathbf{X} + \mathbf{Y} \sim N(\mu_x + \mu_y, \Sigma_x + \Sigma_y)$$

---

## Parameter Estimation

---

Given samples  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$  from a multivariate normal, estimate  $\mu$  and  $\Sigma$ .

### Maximum Likelihood Estimates

**Mean:**

$$\hat{\mu} = (1/n) \sum_i \mathbf{x}_i$$

**Covariance (unbiased):**

$$\hat{\Sigma} = (1/(n-1)) \sum_i (\mathbf{x}_i - \hat{\mu})(\mathbf{x}_i - \hat{\mu})^T$$

## Implementation

```
def fit_multivariate_normal(samples):
    """
    Estimate parameters of multivariate normal from samples

    Parameters:
    -----
    samples : array (n_samples, d)
        Samples from the distribution

    Returns:
    -----
    mu_hat : array (d,)
        Estimated mean vector
    Sigma_hat : array (d, d)
        Estimated covariance matrix
    """
    # Estimate mean
    mu_hat = np.mean(samples, axis=0)

    # Estimate covariance
    Sigma_hat = np.cov(samples.T) # Note: .T transposes

    return mu_hat, Sigma_hat

# Example
true_mu = np.array([6, 6])
true_Sigma = np.array([[1.0, 0.6],
                        [0.6, 1.0]])

# Generate samples
samples = np.random.multivariate_normal(true_mu, true_Sigma, 1000)

# Estimate parameters
mu_hat, Sigma_hat = fit_multivariate_normal(samples)

print("True mean:", true_mu)
print("Estimated mean:", mu_hat)
```

```
print("\nTrue covariance:\n", true_Sigma)
print("Estimated covariance:\n", Sigma_hat)
```

## Using NumPy Built-ins

```
# Mean
mu_hat = np.mean(samples, axis=0)

# Covariance (rowvar=False means each column is a variable)
Sigma_hat = np.cov(samples, rowvar=False)

# Or equivalently (rowvar=True by default, so transpose)
Sigma_hat = np.cov(samples.T)
```

## Verification

```
# With large sample, estimates should be close to true values
n = 100000
samples = np.random.multivariate_normal(true_mu, true_Sigma, n)
mu_hat, Sigma_hat = fit_multivariate_normal(samples)

print("Estimation error (mean):", np.linalg.norm(mu_hat - true_mu))
print("Estimation error (cov):", np.linalg.norm(Sigma_hat - true_Sigma))
```

# Implementation in Python

---

## Creating a Multivariate Normal Distribution

```
from scipy.stats import multivariate_normal

# Parameters
mu = np.array([6, 6])
Sigma = np.array([[1.0, 0.6],
                  [0.6, 1.0]])

# Create distribution object
mvn = multivariate_normal(mean=mu, cov=Sigma)
```

## Sampling

```
# Draw samples
n_samples = 1000
samples = mvn.rvs(size=n_samples) # Shape: (1000, 2)

# Visualize
plt.scatter(samples[:, 0], samples[:, 1], alpha=0.3)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Samples from 2D Normal')
plt.axis('equal')
plt.show()
```

## Computing PDF

```
# At a single point
x = np.array([6, 6])
pdf = mvn.pdf(x)

# At multiple points
points = np.array([[6, 6], [7, 7], [5, 5]])
pdfs = mvn.pdf(points)

# On a grid
x_range = np.linspace(0, 12, 100)
y_range = np.linspace(0, 12, 100)
X, Y = np.meshgrid(x_range, y_range)
pos = np.dstack((X, Y))
Z = mvn.pdf(pos)
```

## Computing Log-Likelihood

```
# Log-likelihood of a single sample
log_likelihood = mvn.logpdf(x)

# Log-likelihood of multiple samples
samples = mvn.rvs(100)
log_likelihoods = mvn.logpdf(samples)

# Total log-likelihood (sum of individual log-likelihoods)
total_log_likelihood = np.sum(log_likelihoods)
```

# Practical Examples

---

## Example 1: Effect of Correlation

```
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

mu = np.array([0, 0])

# Different correlations
correlations = [-0.9, -0.5, 0.0, 0.5, 0.9]

fig, axes = plt.subplots(1, 5, figsize=(20, 4))

for i, rho in enumerate(correlations):
    Sigma = np.array([[1.0, rho],
                      [rho, 1.0]])

    mvn = multivariate_normal(mu, Sigma)
    samples = mvn.rvs(500)

    axes[i].scatter(samples[:, 0], samples[:, 1], alpha=0.3)
    axes[i].set_title(f' $\rho = \{rho\}$ ')
    axes[i].set_xlim(-3, 3)
    axes[i].set_ylim(-3, 3)
    axes[i].axis('equal')

plt.tight_layout()
plt.show()
```

## Example 2: Effect of Variance

```
mu = np.array([0, 0])

# Different variances
variances = [0.5, 1.0, 2.0, 4.0]

fig, axes = plt.subplots(1, 4, figsize=(16, 4))

for i, var in enumerate(variances):
    Sigma = np.array([[var, 0],
                      [0, var]])

    mvn = multivariate_normal(mu, Sigma)
    samples = mvn.rvs(500)

    axes[i].scatter(samples[:, 0], samples[:, 1], alpha=0.3)
    axes[i].set_title(f' $\sigma^2 = \{var\}$ ')
    axes[i].set_xlim(-6, 6)
    axes[i].set_ylim(-6, 6)
    axes[i].axis('equal')

plt.tight_layout()
plt.show()
```

## Example 3: Fitting to Data

```
# Suppose we have submarine observation data
observations = np.loadtxt('data/submarine.txt')

# Fit multivariate normal
mu_hat = np.mean(observations, axis=0)
Sigma_hat = np.cov(observations.T)

print(f"Estimated mean: {mu_hat}")
print(f"Estimated covariance:\n{Sigma_hat}")

# Create fitted distribution
fitted_mvn = multivariate_normal(mu_hat, Sigma_hat)

# Visualize fit
plt.scatter(observations[:, 0], observations[:, 1],
            alpha=0.3, label='Observations')

# Draw samples from fitted distribution
samples = fitted_mvn.rvs(250)
plt.scatter(samples[:, 0], samples[:, 1],
            alpha=0.3, color='red', marker='x',
            label='Samples from fitted model')

plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.title('Fitting Multivariate Normal to Data')
plt.show()
```

## Example 4: Log-Likelihood for Model Comparison

```
# Compare two models
# Model 1: General covariance
mu1 = np.mean(observations, axis=0)
Sigma1 = np.cov(observations.T)
mvn1 = multivariate_normal(mu1, Sigma1)

# Model 2: Diagonal covariance (assumes independence)
mu2 = np.mean(observations, axis=0)
Sigma2 = np.diag(np.var(observations, axis=0))
mvn2 = multivariate_normal(mu2, Sigma2)

# Compute log-likelihoods
llik1 = np.sum(mvn1.logpdf(observations))
llik2 = np.sum(mvn2.logpdf(observations))

print(f"Log-likelihood (full covariance): {llik1:.2f}")
print(f"Log-likelihood (diagonal): {llik2:.2f}")

if llik1 > llik2:
    print("Full covariance model fits better!")
else:
    print("Diagonal model fits better!")
```

# Common Mistakes

---

## Mistake 1: Confusing row/column vectors

```
# WRONG: mean should be 1D array
mu = np.array([[6], [6]]) # Shape: (2, 1) - column vector

# CORRECT
mu = np.array([6, 6]) # Shape: (2,)
```

## Mistake 2: Non-symmetric covariance

```
# WRONG: must be symmetric!
Sigma = np.array([[1.0, 0.6],
                  [0.5, 1.0]]) #  $\sigma_{12} \neq \sigma_{21}$ 

# CORRECT
Sigma = np.array([[1.0, 0.6],
                  [0.6, 1.0]])
```

## Mistake 3: Not positive definite

```
# WRONG: not positive definite!
Sigma = np.array([[1.0, 2.0],
                  [2.0, 1.0]]) #  $|\rho| > 1$ 

# Check positive definiteness
eigenvalues = np.linalg.eigvals(Sigma)
print("Eigenvalues:", eigenvalues) # Should all be positive!

# CORRECT: ensure  $|\rho| \leq 1$ 
Sigma = np.array([[1.0, 0.9],
                  [0.9, 1.0]])
```

## Mistake 4: Wrong axis in np.cov

```

samples = np.random.multivariate_normal([0, 0], [[1, 0], [0, 1]], 100)
# Shape: (100, 2) - 100 samples of 2D vectors

# WRONG
Sigma_wrong = np.cov(samples) # Treats each row as a variable!

# CORRECT (specify rowvar=False)
Sigma_correct = np.cov(samples, rowvar=False)

# Or equivalently
Sigma_correct = np.cov(samples.T)

```

## Summary

### Multivariate Normal Distribution:

$$X \sim N(\mu, \Sigma)$$

$$f(x) = (1/\sqrt{((2\pi)^d |\Sigma|)}) \exp(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu))$$

### Parameters:

- $\mu$  (d×1): Mean vector
- $\Sigma$  (d×d): Covariance matrix (symmetric, positive semi-definite)

### Key Properties:

1. Marginals are normal
2. Linear combinations are normal
3. Independence  $\iff$  zero covariance
4. Conditionals are normal
5. Completely specified by  $\mu$  and  $\Sigma$

**Parameter Estimation** (from samples):

```
mu_hat = np.mean(samples, axis=0)
Sigma_hat = np.cov(samples.T)
```

**Python Implementation:**

```
from scipy.stats import multivariate_normal

mvn = multivariate_normal(mean=mu, cov=Sigma)
samples = mvn.rvs(n) # Sample
pdf = mvn.pdf(x) # Evaluate PDF
llik = mvn.logpdf(x) # Log-likelihood
```

**Common Use Cases:**

- Modeling correlated variables
- Maximum likelihood estimation
- Bayesian inference
- Gaussian processes
- Kalman filtering
- Principal Component Analysis (PCA)

**Remember:**

- Covariance matrix must be symmetric and positive definite
- Correlation coefficient:  $-1 \leq \rho \leq 1$
- Use `np.cov(samples.T)` or `np.cov(samples, rowvar=False)`
- Work in log-space for numerical stability